Lab 14 — ConfigMaps and Secrets

Theory — how the same image serves every environment in a cluster, where to put a password when `-e` and `--env-file` no longer exist, and why "base64" has never protected anyone.

Objectives

- Carry the lab 08 principle — one image, injected configuration — over to the Kubernetes objects: **ConfigMap** and **Secret**.
- Inject configuration as **environment variables** or as **mounted files**, and know when to choose which.
- Understand what a Secret really protects, and what it does not.
- Know what happens when a ConfigMap is modified: variables frozen, files updated.
- Relate all of this to Spring Boot profiles and to enterprise secrets managers.

---

1. The lab 08 problem, at cluster scale

In lab 08, the rule was: an immutable image, configuration coming from outside, secrets in files rather than variables. The mechanisms were `ENV`, `-e`, `--env-file`, `--secret`. In the cluster, there is no command line any more: everything is a manifest. And a Pod template (lab 12) is **the same for every replica and every environment** — hard-coding `APP_PROFILE: prod` in it would amount to rebuilding one image per environment.

Kubernetes therefore separates configuration from the template into two objects: the **ConfigMap** for what is not sensitive (URLs, profiles, configuration files), the **Secret** for what grants access (passwords, tokens, certificates). The template **references** them by name; each environment has its own, and the Deployment does not change from one namespace to the next.

> **→ DOCKER RECAP**
> ConfigMap = `--env-file` and the configuration files bind-mounted; Secret = `podman secret` and the `_FILE` convention. The difference: in the cluster, these objects are stored in the API server, versioned with the manifests, and available on whichever node the Pod lands on — no more `hostPath` to a file that only exists on your machine (lab 11, question 13).

2. The ConfigMap: keys, values, files

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
data:
  APP_MESSAGE: "bonjour depuis une ConfigMap"
  APP_PROFILE: "staging"
```

A ConfigMap is a dictionary of strings. A value can be a word (`staging`) or an entire file (`default.conf` with its twenty lines): `kubectl create configmap web-nginx --from-file=default.conf=nginx-proxy.conf` creates the key `default.conf` with the file's content. It is only useful once **referenced** by a Pod, in two ways:

Injection	Syntax in the template	What the container sees	ConfigMap update
environment variables	`envFrom: [configMapRef: {name: api-config}]` (all keys) or `env: [valueFrom: configMapKeyRef]` (one key)	`APP_PROFILE=staging` in `env`	**not propagated**: the Pod must be recreated
volume	`volumes: [configMap: {name: web-nginx}]` + `volumeMounts`	one file per key in the mounted directory	**propagated** in about a minute, with no restart

The choice follows lab 08: variables for what the application reads at startup (Spring Boot, the JVM), files for structured or long configuration (`nginx.conf`, `application.yaml`, a certificate). In the hands-on lab, nginx is the **unmodified** official `nginx:alpine` image, configured by two mounted ConfigMaps — the `web:1.0` image from lab 13 is no longer needed.

PITFALL

An environment variable is copied into the process at creation: modifying the ConfigMap changes **nothing** for running Pods. `kubectl rollout restart deployment/api` recreates them with the new values. A mounted file, on the other hand, is a symbolic link into a directory the kubelet swaps during its periodic sync (`..data/`, visible in `ls -l`) — unless the mount uses `subPath`, which copies the file once and for all. And a referenced key that does not exist (`key: profil` instead of `APP_PROFILE`) blocks the Pod in `CreateContainerConfigError`: `couldn't find key profil in ConfigMap`.

→ SPRING BOOT

The two routes combine naturally: a ConfigMap of variables (`SPRING_PROFILES_ACTIVE`, `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`) and a file ConfigMap mounted at `/config/application.yaml`, which Spring Boot reads with no extra configuration. For secrets, `spring.config.import=optional:configtree:/run/secrets/` turns every mounted file into a property (`db.password` from `/run/secrets/db.password`). Precedence remains that of lab 08: variable > external file > file inside the JAR.

3. The Secret: the same thing, with a "sensitive" label

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  POSTGRES_PASSWORD: "S3cr3t-pg"
```

A Secret is written like a ConfigMap (`stringData` for text, `data` for base64) and injected the same way: `env` with `secretKeyRef`, or a `secret:` volume that mounts one file per key. What it adds: the values are **base64-encoded** in the API (`UzNjcjN0LXBn` for `S3cr3t-pg`), the kubelet mounts them in **tmpfs** (never on the node's disk), RBAC permissions (lab 16) can forbid `get secrets` to whoever may read ConfigMaps, and tools (`kubectl describe`, dashboards) mask them.

→ SECURITY

Base64 is **not** encryption: `kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d` hands the password to anyone who can read the Secret. In etcd, Secrets are in plain text by default; serious clusters enable **encryption at rest** (`EncryptionConfiguration`, or a cloud KMS) and restrict access to etcd. A `Secret` manifest with its value **never goes into Git**: it goes in encrypted (SOPS, Sealed Secrets), or is generated by a controller from an external manager (External Secrets Operator + Vault, AWS Secrets Manager…). The lab 08 rule — a secret visible in a ticket is compromised — applies to a `kubectl get -o yaml` pasted into a chat.

Secrets have types: `Opaque` (free-form), `kubernetes.io/tls` (certificate + key, consumed by the Ingress for HTTPS), `kubernetes.io/dockerconfigjson` (registry credentials, referenced by `imagePullSecrets` to pull a private image — which our `k3s ctr images import` spares us). A Secret, like a ConfigMap, can be marked `immutable: true`: no longer modifiable, hence no risk of accidental change, and less load on the API server.

4. Four ways to create

Command	Use
`kubectl apply -f configmap.yaml`	the normal route: a versioned file
`kubectl create configmap x --from-literal=K=v`	a quick test
`kubectl create configmap x --from-file=key=file`	a whole file under one key
`kubectl create secret generic x --from-literal=… --dry-run=client -o yaml`	**produce** the manifest (with the base64 computed) without creating it, to encrypt it afterwards

`--from-env-file=api.env` reads a `KEY=value` file from lab 08: the move from Compose to Kubernetes happens without rewriting the configuration.

5. What changes between environments

The Deployment references `api-config` and `db-secret` by **name**; every namespace (or every cluster) owns its own objects of that name, with its own values. Promoting an image from staging to production (lab 10) consists of applying the **same** Deployment in a namespace where the ConfigMaps and Secrets say `prod`. Kustomize and Helm (lab 17) generate those variants; the underlying mechanism stays this one.

K3S

Nothing specific: ConfigMaps and Secrets are in the core of the API, and k3s stores Secrets in plain text in SQLite just as a bare cluster stores them in etcd. k3s's `--secrets-encryption` option enables encryption at rest; a managed cluster offers it as a checkbox. On your workstation, that is not the point: the `k3s.db` database is yours.

6. In the workplace

- **One ConfigMap per application and per environment**, versioned with the manifests; the values that differ (`SPRING_PROFILES_ACTIVE`, database URL, log levels) live there, never in the template.
- **Secrets are generated, not written**: External Secrets Operator or an equivalent syncs them from the company's secrets manager; the Git repository holds only the *reference* ("the

password of the `app` database in prod").

- **Files for secrets** (`volume: secret`, `_FILE` / `configtree` convention), variables for configuration: the same discipline as lab 08, because `kubectl exec … env` shows the whole environment.
- **Rotation**: a Secret mounted as a volume updates with no restart, provided the application re-reads the file; a Secret in a variable requires a `rollout restart`. That is a design criterion, not a detail.
- **RBAC**: reading a namespace's ConfigMaps is common; reading its Secrets is restricted.

Remember

- ConfigMap = non-sensitive configuration, Secret = access; both are dictionaries referenced by the template, by name.
- Variables (`envFrom`, `valueFrom`) for what is read at startup; volumes for configuration files and secrets.
- Modifying a ConfigMap does not change running Pods' variables (`rollout restart`); a mounted file updates by itself, except with `subPath`.
- A missing key blocks the Pod: `CreateContainerConfigError`, `couldn't find key`.
- Base64 is not encryption; a Secret is protected by RBAC, encryption at rest, and by never being committed in plain text.
- `--dry-run=client -o yaml` produces a manifest; `--from-env-file` reuses an `api.env` from lab 08.
- Same Deployment everywhere, different ConfigMaps and Secrets per namespace: that is how an image is promoted.

Vocabulary

ConfigMap: dictionary of non-sensitive configuration. — **Secret**: dictionary of sensitive values, base64-encoded, mounted in tmpfs. — **`envFrom`** / **`valueFrom`**: injection of all keys / of one key as a variable. — **`configMap`** / **`secret`** **volume**: injection as files, one per key. — **`subPath`**: mount of a single file, with no automatic update. — **`CreateContainerConfigError`**: referenced key or object not found. — **`stringData`** / **`data`**: plain / base64 values in a Secret. — **`immutable`**: object that cannot be modified. — **encryption at rest**: protection of Secrets in etcd. — **External Secrets / Sealed Secrets / SOPS**: tools for never committing a secret in plain text.